

2026 开源之夏项目申请书：为 Excelize 添加获取形状支持

基本信息

- 项目名称：为 Excelize 添加获取形状支持 (Add get shapes support for Excelize)
- 申请人：常奥
- 联系邮箱：ca879112685123@163.com
- GitHub 主页：<https://github.com/wangyica879112>
- 项目导师：淡紫色的风鹏 (414326615@qq.com)
- 目标项目：Excelize (<https://github.com/xuri/excelize.git>)

一、项目调研 (Project Research)

1.1 项目背景

Excelize 是目前 Go 语言中最受欢迎的、用于操作 Office Excel 文档（OOXML 格式）的基础开源库。目前，Excelize 已经具备了强大的写入能力，支持在工作表中添加 187 种原生 Excel 形状（Shape）。然而，随着用户使用场景的丰富，针对已有 Excel 文档中形状的读取需求日益增加。

1.2 需求分析

本课题的核心目标是实现 GetShapes 函数。根据 Issue #436 及项目要求，该函数需要：

- 输入参数：工作表名称 (Worksheet Name) 和 单元格坐标 (Cell Coordinates，如 "A1")。
- 输出结果：该单元格内的形状列表，包含形状名称和图形属性信息（如颜色、大小、样式等）。
- 技术要求：基于 ISO/IEC 29500 国际标准，解析工作表对应的 DrawingML (XML) 文件，精准提取形状信息。

1.3 现状及痛点分析

目前社区用户无法直接通过 Excelize 获取工作表内已存在的形状信息。OOXML 标准中，形状信息并不直接存在于 Worksheet 的 XML 中，而是通过 Relationship 关联到独立的 xl/drawings/drawing[N].xml 文件。解析过程涉及复杂的命名空间（如 xdr, a）和不同的锚点类型（twoCellAnchor, oneCellAnchor, absoluteAnchor），这需要对底层的 XML 结构进行精准映射和高效查找。

二、设计方案 (Design Plan)

2.1 接口与数据结构设计

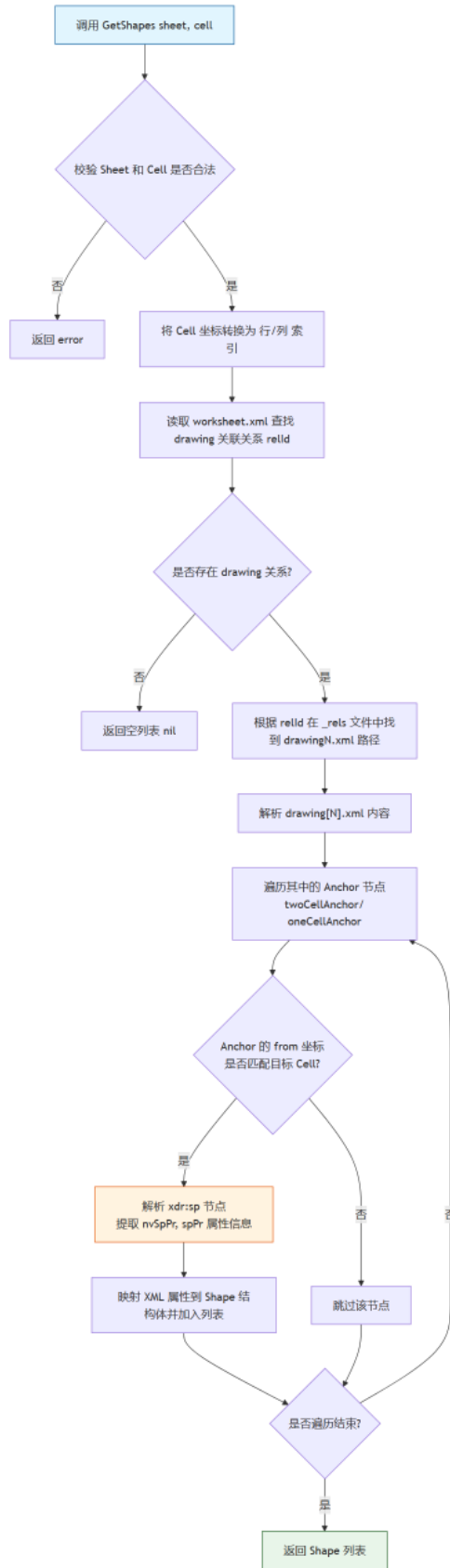
为了保持 Excelize API 的一致性与易用性，将在 shape.go 中定义如下核心接口及相关结构体扩展：

// GetShapes 提供根据工作表名称和单元格坐标获取形状列表的功能

```
func (f *File) GetShapes(sheet, cell string) ([]Shape, error) {  
    // ... 核心实现逻辑  
}
```

2.2 系统流程图 (Flowchart)

解析一个单元格中的形状，需要经历坐标转换、关系映射、XML 解析和属性过滤四个主要阶段，核心流程图如下：



三、实现方案 (Implementation Plan)

3.1 核心技术栈

- 开发语言：Go
- 涉及技术：Go 标准库 `encoding/xml`, `archive/zip`, 以及核心的 XML 序列化与反序列化算法。
- 规范参考：ISO/IEC 29500-1:2016 (Part 1: Fundamentals And Reference Material) - Section 20 (DrawingML).

3.2 核心步骤实现

1. 工作表与 DrawingML 关系解析：复用 Excelize 现有的内部方法获取 sheet 的 drawing 关系 ID，并解析 `xl/worksheets/_rels/sheet[N].xml.rels` 文件定位到对应的绘画文件（如 `xl/drawings/drawing1.xml`）。
2. XML 数据结构反序列化：在现有的 `xmlDrawing` 或相关结构体基础上，补充或完善对应 `xdr:sp` (Shape), `xdr:nvSpPr` (Non-Visual Shape Drawing Properties), `xdr:spPr` (Visual Shape Properties) 的 Go tag 映射，以便于通过 `xml.Unmarshal` 一次性读取数据。
3. 坐标匹配与过滤：解析 `<xdr:from>` 节点中的 `<xdr:col>` 和 `<xdr:row>`（注意：XML 中以 0 为起始索引，需与输入的 Excel 坐标如 "A1" 进行换算），过滤出匹配目标单元格的形状节点。
4. 属性组装：将读取到的 187 种预设几何形状类型 (`prstGeom`)、颜色填充 (`solidFill` / `gradFill`)、线条样式 (`ln`) 等底层结构，转换为对用户友好的 Shape 结构体字段。

3.3 难点及应对策略

- 难点 1：性能问题
- 策略：如果评估发现 `xml.Unmarshal` 导致内存占用过大，将采用 `xml.Decoder` 进行基于 Token 的流式解析 (Streaming parsing)，在匹配到非目标单元格的 Anchor 时直接跳过 (Skip)，保障库的高性能表现。
- 难点 2：兼容性与异常容错

- 策略: 使用松散的结构体映射, 并针对缺失属性提供符合 ISO 标准的默认值; 增强错误捕捉机制。

四、测试方案 (Testing Plan)

在 `shape_test.go` 中编写完备的测试代码, 确保增量代码的行测试覆盖率达到 100%。

4.1 测试用例设计

- 正常路径测试:
 - 包含单个形状的单元格读取。
 - 包含多个形状 (堆叠) 的单元格读取。
 - 获取并校验各种典型形状 (矩形、箭头、星形等) 的属性细节是否准确无误。
- 异常及边界测试 (Edge Cases):
 - 传入不存在的 `sheet` 名称或非法的 `cell` 坐标 (如 "A-1")。
 - 目标单元格内没有任何形状 (应返回空切片和 `nil` 错误)。
 - 工作表完全没有包含任何 `Drawing` 关联文件。
 - 损坏或不规范的 XML 内部结构 (测试解析器的容错性)。
- 不同锚点类型的兼容测试:
 - 构造包含 `twoCellAnchor` 和 `oneCellAnchor` 的 Excel 文件进行读取比对。

4.2 性能测试 (Benchmark)

- 使用 `go test -bench` 编写基准测试, 验证 `GetShapes` 在面临包含 1,000 / 10,000 个形状的极限文档时的执行时间与内存分配 (Allocs) 情况, 确保符合 "High-quality code with efficient performance" 的要求。

五、开发计划 (Development Schedule)

项目开发周期为 8 月 15 日至 11 月 15 日, 详细排期如下:

时间节点	阶段	具体任务与交付
08.15 - 08.31	规范研读与方案细化	1. 深入研读 ISO/IEC 29500 DrawingML 章节。 2. 熟悉 excelize 的源码架构，特别是 shape.go 和 XML 结构定义。 3. 在本地搭建好测试床，构造含有多种形状的 Excel 测试文件。 里程碑： 提交详细的技术设计方案并与导师 Review。
09.01 - 09.30	核心功能开发	1. 实现坐标转换、关系解析逻辑。 2. 在 shape.go 中添加 GetShapes 核心代码。 3. 完善内部用于反序列化 DrawingML 的 XML 映射结构体。 4. 完成坐标匹配过滤功能。 里程碑： 能够成功读取指定单元格内的基本形状信息。
10.01 - 10.15	属性完善与全面适配	1. 完善 187 种形状属性（颜色、边框、尺寸等）的提取与映射转换。 2. 处理 oneCellAnchor 和 twoCellAnchor 等不同类型的差异。 里程碑： 功能完全实现，达到 Issue #436 的全部要求。
10.16 - 10.31	测试覆盖与性能优化	1. 在 shape_test.go 中补充全面的单元测试。 2. 通过 go test -cover 确保增量代码覆盖率 100%。

		3. 运行 Benchmark 测试，若有性能瓶颈则进行流式解析优化。 里程碑： 提交完整的 Pull Request (PR)。
11.01 - 11.15	代码审查与结项	1. 配合导师和 Maintainer 意见进行 Code Review 修复。 2. 补充中英文档和 Godoc 注释。 3. 编写开源之夏结项报告。 里程碑： PR 成功合并入主干，顺利结项。

六、参考资料 (References)

1. Excelize GitHub Repository: <https://github.com/xuri/excelize.git>
2. Target Issue #436: [Need to support get shapes in sheet](#)
3. Standard Specification: ISO/IEC 29500-1:2016 (Information technology — Document description and processing languages — Office Open XML File Formats)
4. Go 官方文档: encoding/xml - <https://pkg.go.dev/encoding/xml>